

✦ N E X U S   B O A R D

# NEXUS

## Foro Gaming Competitivo

Plataforma web full-stack para una comunidad de videojuegos competitivos: foros por categoría, sistema de likes en tiempo real, panel de administración completo y copias de seguridad de base de datos y archivos.

### Documentación y presentación técnica del proyecto

---

• PHP 8

• MySQL · InnoDB

• JavaScript · AJAX

• HTML5 · CSS3

• API REST

• XAMPP

---

**Hugo Reche**

Proyecto Intermodular · Curso académico 2025-2026

DEDICACIÓN TOTAL

**48,5 h**

# Índice

---

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| <b>01</b> | <b>Presentación del proyecto</b>        | <b>4</b>  |
|           | Contexto, objetivo, público y alcance   |           |
| <b>02</b> | <b>Objetivos y requisitos</b>           | <b>6</b>  |
|           | Requisitos funcionales y no funcionales |           |
| <b>03</b> | <b>Planificación y metodología</b>      | <b>8</b>  |
|           | Tablero Kanban y registro de 48,5 horas |           |
| <b>04</b> | <b>Arquitectura y stack tecnológico</b> | <b>11</b> |
|           | Modelo cliente-servidor sobre XAMPP     |           |
| <b>05</b> | <b>Modelo de datos</b>                  | <b>13</b> |
|           | Base de datos usuarios_db · 3 tablas    |           |
| <b>06</b> | <b>Estructura de archivos</b>           | <b>16</b> |
|           | Organización del código fuente          |           |
| <b>07</b> | <b>Backend y API REST</b>               | <b>18</b> |
|           | api.php · 24 acciones AJAX              |           |
| <b>08</b> | <b>Autenticación y seguridad</b>        | <b>23</b> |
|           | Hash, sesiones, CSRF y protección       |           |

# Índice *(cont.)*

---

|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| <b>09</b> | <b>Funcionalidades principales</b>            | <b>26</b> |
|           | Foros, likes, búsqueda y comunidad            |           |
| <b>10</b> | <b>Panel de administración</b>                | <b>29</b> |
|           | Resumen, moderación, usuarios y backups       |           |
| <b>11</b> | <b>Copias de seguridad y restauración</b>     | <b>33</b> |
|           | SQL, ZIP y restauración con PIN               |           |
| <b>12</b> | <b>Frontend: HTML, CSS y JavaScript</b>       | <b>35</b> |
|           | Estética Cosmos y arquitectura del cliente    |           |
| <b>13</b> | <b>Experiencia de usuario y accesibilidad</b> | <b>39</b> |
|           | Responsive, atajos y motion                   |           |
| <b>14</b> | <b>Despliegue y puesta en marcha</b>          | <b>41</b> |
|           | Scripts .bat, red local y ngrok               |           |
| <b>15</b> | <b>Pruebas y validación</b>                   | <b>44</b> |
|           | Casos de prueba y depuración                  |           |
| <b>16</b> | <b>Conclusiones y mejoras futuras</b>         | <b>46</b> |
|           | Resultados y líneas de trabajo                |           |
| <b>—</b>  | <b>Anexos</b>                                 | <b>48</b> |
|           | Credenciales, comandos y glosario             |           |

# Presentación del proyecto

**NEXUS** es un foro web para una comunidad de videojuegos competitivos. Permite a los usuarios registrarse, crear hilos de debate organizados por categorías, publicar mensajes, dar “me gusta” a las publicaciones y consultar un ranking de la comunidad. Incluye además un completo panel de administración para moderar contenido, gestionar usuarios y realizar copias de seguridad de la base de datos y los archivos.

## 1.1 Contexto académico

Este proyecto se ha desarrollado como **Proyecto Intermodular** del curso 2025–2026. Su finalidad es integrar en una sola aplicación los conocimientos de varios módulos: bases de datos, programación de servidor (PHP), desarrollo de interfaces web (HTML, CSS y JavaScript), despliegue de aplicaciones y documentación técnica. El resultado es una aplicación web *full-stack* funcional que cubre el ciclo completo de vida del software: análisis, diseño, desarrollo, pruebas, despliegue y mantenimiento.



### Objetivo

Construir una aplicación web full-stack funcional sobre PHP y MySQL, cubriendo análisis, diseño, desarrollo, pruebas, despliegue y mantenimiento.



### Público

Comunidad de jugadores que comparten estrategias, noticias y debates sobre títulos de FPS, MOBA, estrategia y hardware gaming.



### Alcance

Autenticación segura, foros por categoría, likes vía AJAX, panel de administración, estadísticas, moderación y backups en formato SQL y ZIP.



### Identidad

Diseño con estética espacial (*void / starfield*), interfaz responsive y experiencia de usuario centrada en la rapidez y la legibilidad.

## Categorías del foro

● FPS

● MOBA

● Estrategia

● Noticias

● Hardware

● General

## 1.2 Problema que resuelve

Las comunidades de jugadores suelen dispersarse entre chats efímeros (Discord), redes sociales generalistas y foros saturados de ruido. NEXUS propone un espacio **centrado en el debate de calidad**: hilos organizados por temática, un sistema de reputación basado en likes, moderación activa y un tono “serio” orientado al juego competitivo. El lema del proyecto resume esa intención: *“Juega en serio. Piensa más rápido.”*

## 1.3 Pilares del proyecto

### COMUNIDAD

#### Participación

Registro abierto, publicación de hilos, likes y ranking de miembros que premia la aportación.

### CONTROL

#### Moderación

Cada publicación de un miembro pasa por aprobación. El administrador modera, gestiona roles y banea.

### ROBUSTEZ

#### Mantenimiento

Backups completos de datos y archivos, y restauración del sitio desde un ZIP previo.

## 1.4 Resumen de capacidades

**24**

ACCIONES API

**3**

TABLAS BBDD

**6**

CATEGORÍAS

**2**

ROLES

**5**

PESTAÑAS ADMIN

### En una frase

NEXUS es un foro gaming full-stack en PHP 8 + MySQL, con API REST propia, interfaz responsive de estética espacial y un panel de administración con moderación, estadísticas y sistema de copias de seguridad protegido por PIN.

# Objetivos y requisitos

El proyecto parte de un objetivo general que se descompone en objetivos específicos y se concreta en una lista de requisitos funcionales (qué debe hacer el sistema) y no funcionales (cómo debe comportarse).

## 2.1 Objetivo general

Desarrollar y desplegar una aplicación web de foro para comunidades gaming que sea segura, usable, mantenible y completamente funcional en un entorno XAMPP local con posibilidad de acceso en red local e internet.

## 2.2 Objetivos específicos

- › **Autenticación:** registro y login seguros con cifrado de contraseñas y gestión de sesión.
- › **Contenido:** permitir crear, listar, filtrar, buscar y ordenar hilos por categoría.
- › **Interacción:** sistema de likes en tiempo real sin recargar la página (AJAX).
- › **Administración:** panel para moderar, gestionar usuarios y ver estadísticas.
- › **Persistencia:** base de datos relacional normalizada con integridad referencial.
- › **Mantenimiento:** copias de seguridad y restauración del sitio.
- › **Despliegue:** arranque automatizado y acceso desde otros dispositivos de la red.

## 2.3 Requisitos funcionales

| ID    | REQUISITO                                                              | PRIORIDAD |
|-------|------------------------------------------------------------------------|-----------|
| RF-01 | El visitante puede registrarse con nombre, nick, email y contraseña.   | Alta      |
| RF-02 | El usuario registrado puede iniciar y cerrar sesión.                   | Alta      |
| RF-03 | El usuario puede crear hilos eligiendo una categoría.                  | Alta      |
| RF-04 | Cualquier visitante puede leer los hilos aprobados.                    | Alta      |
| RF-05 | Se pueden dar likes a las publicaciones vía AJAX.                      | Media     |
| RF-06 | El listado se puede filtrar por categoría, buscar y ordenar.           | Media     |
| RF-07 | Existe un ranking (leaderboard) de miembros por likes.                 | Media     |
| RF-08 | El visitante puede enviar mensajes mediante un formulario de contacto. | Media     |
| RF-09 | El administrador aprueba o rechaza publicaciones pendientes.           | Alta      |
| RF-10 | El administrador gestiona roles y elimina usuarios.                    | Alta      |
| RF-11 | El administrador descarga y restaura copias de seguridad.              | Media     |

## 2.4 Requisitos no funcionales

| ID     | CATEGORÍA      | REQUISITO                                                                |
|--------|----------------|--------------------------------------------------------------------------|
| RNF-01 | Seguridad      | Las contraseñas se almacenan cifradas con bcrypt (nunca en texto plano). |
| RNF-02 | Seguridad      | Las consultas usan sentencias preparadas para evitar inyección SQL.      |
| RNF-03 | Seguridad      | Las acciones destructivas requieren rol admin y un PIN adicional.        |
| RNF-04 | Rendimiento    | Compresión gzip y caché de navegador mediante <code>.htaccess</code> .   |
| RNF-05 | Usabilidad     | Interfaz responsive adaptada a móvil, tablet y escritorio.               |
| RNF-06 | Accesibilidad  | Soporte de <code>prefers-reduced-motion</code> y alto contraste.         |
| RNF-07 | Mantenibilidad | Código organizado en archivos con responsabilidad única.                 |
| RNF-08 | Portabilidad   | Funciona sobre XAMPP estándar sin configuración manual de la BBDD.       |

## 2.5 Actores del sistema



### Visitante

Lee hilos, busca, consulta perfiles y el ranking, y envía mensajes de contacto. No puede publicar.



### Miembro

Visitante registrado. Publica hilos (sujetos a aprobación), da likes y aparece en el ranking.



### Administrador

Modera, gestiona usuarios y roles, ve estadísticas y realiza backups y restauraciones.

## 2.6 Caso de uso destacado: publicar un hilo

- El miembro abre el compositor desde el foro e introduce título, contenido y categoría.
- El cliente valida longitudes mínimas y envía la petición `add_post` por AJAX.
- El servidor sanea los datos, comprueba la sesión y guarda el hilo como *pendiente*.
- El administrador lo revisa en la pestaña **Moderación** y lo aprueba o rechaza.
- Una vez aprobado, el hilo aparece públicamente en el listado del foro.

### Regla de negocio

Los hilos creados por un administrador se publican directamente ( `approved = 1` ); los de un miembro quedan pendientes hasta su aprobación.

# Planificación y metodología

El desarrollo se organizó con una metodología ágil basada en un **tablero Kanban**, dividiendo el trabajo en 10 módulos y 32 tareas. El esfuerzo total registrado fue de **48,5 horas**, con una media de 1,5 horas por tarea.

|               |           |           |               |
|---------------|-----------|-----------|---------------|
| <b>48,5 h</b> | <b>10</b> | <b>32</b> | <b>1,5 h</b>  |
| HORAS TOTALES | MÓDULOS   | TAREAS    | MEDIA / TAREA |

## 3.1 Metodología de trabajo

Se adoptó un enfoque incremental: cada módulo del Kanban representa una funcionalidad completa que se desarrolla, prueba y se da por cerrada antes de pasar a la siguiente. Las columnas del tablero se corresponden con los grandes bloques del proyecto, y cada tarjeta registra las horas dedicadas. Este control de horas permite analizar dónde se concentró el esfuerzo y justificar la dedicación total.

## 3.2 Tablero Kanban — módulos (1 de 2)

|                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Análisis y Diseño</b> <b>4,5 h</b> <ul style="list-style-type: none"> <li>Definición de requisitos y alcance 1 h</li> <li>Diseño del modelo de datos (BBDD) 1,5 h</li> <li>Wireframes / mockups de la interfaz 2 h</li> </ul>                                  | <b>Configuración del Entorno</b> <b>5,5 h</b> <ul style="list-style-type: none"> <li>Instalación de XAMPP (Apache + MySQL) 1,5 h</li> <li>Organización de archivos en htdocs 1 h</li> <li>phpMyAdmin y creación de la BBDD 1 h</li> <li>Conexión a base de datos 1 h</li> <li>Estructura del proyecto y dependencias 1 h</li> </ul> |
| <b>Autenticación de Usuarios</b> <b>6 h</b> <ul style="list-style-type: none"> <li>Formulario y lógica de registro 2 h</li> <li>Login y gestión de sesión 2 h</li> <li>Cifrado de contraseñas (hash) 1 h</li> <li>Validaciones y mensajes de error 1 h</li> </ul> | <b>Sistema de Foros</b> <b>9 h</b> <ul style="list-style-type: none"> <li>Categorías y subforos 2 h</li> <li>Crear y listar hilos (threads) 3 h</li> <li>Publicar y mostrar mensajes 2,5 h</li> <li>Paginación y orden de mensajes 1,5 h</li> </ul>                                                                                 |



### 3.3 Tablero Kanban — módulos (2 de 2)

#### Sistema de Likes

3 h

|                            |       |
|----------------------------|-------|
| Modelo de likes en la BBDD | 1 h   |
| Dar / quitar like (AJAX)   | 1,5 h |
| Contador de likes dinámico | 0,5 h |

#### Panel de Administración

7,5 h

|                                       |       |
|---------------------------------------|-------|
| Vista principal del panel (dashboard) | 1,5 h |
| Gestión de usuarios (roles / baneos)  | 2 h   |
| Moderación de hilos y mensajes        | 2 h   |
| Estadísticas del foro                 | 2 h   |

#### PHP, HTML, JS y CSS

6 h

|                                |       |
|--------------------------------|-------|
| Estructura HTML y vistas (PHP) | 2 h   |
| Maquetación y CSS responsive   | 2,5 h |
| Interactividad con JavaScript  | 1,5 h |

#### Backups y Mantenimiento

3,5 h

|                                |       |
|--------------------------------|-------|
| Copias de seguridad de la BBDD | 1,5 h |
| Exportación / importación SQL  | 1 h   |
| Backup de archivos (htdocs)    | 1 h   |

#### Pruebas y Depuración

2,5 h

|                     |       |
|---------------------|-------|
| Pruebas funcionales | 1,5 h |
| Corrección de bugs  | 1 h   |

#### Despliegue y Documentación

1 h

|                            |     |
|----------------------------|-----|
| Documentación del proyecto | 1 h |
|----------------------------|-----|

### 3.4 Distribución de horas por módulo

El **Sistema de Foros** (9 h) y el **Panel de Administración** (7,5 h) concentran el mayor esfuerzo, al ser el núcleo funcional del proyecto. La configuración del entorno y la autenticación también requirieron dedicación significativa por su carácter crítico.



### 3.5 Lectura de la planificación

#### Esfuerzo de desarrollo

Los módulos puramente de programación (foros, autenticación, likes, admin, frontend) suman **31,5 h**, el 65% del total.

#### Esfuerzo de soporte

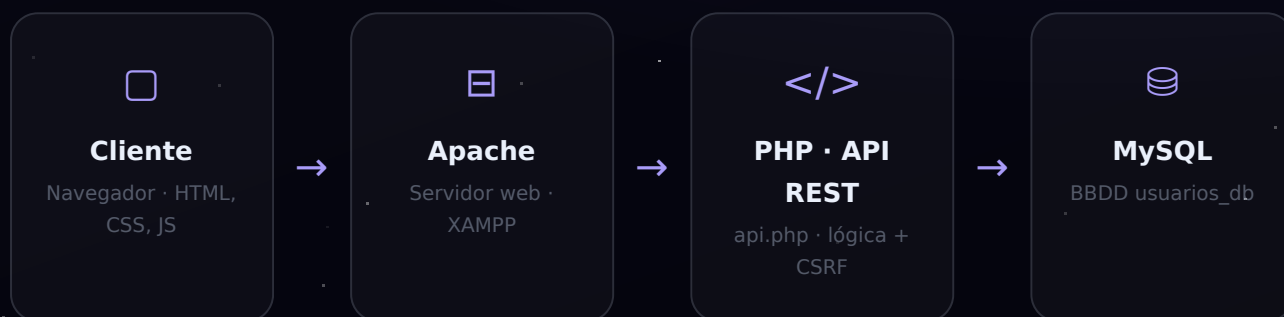
Análisis, entorno, pruebas, backups y documentación suman **17 h**, el 35% restante: la base que sostiene el desarrollo.

#### Conclusión de la planificación

La distribución confirma que el valor del proyecto está en las funcionalidades de comunidad y administración, mientras que un tercio del tiempo se invirtió en cimientos (entorno, datos, pruebas) imprescindibles para la calidad final.

# Arquitectura y stack tecnológico

Arquitectura **cliente-servidor** sobre XAMPP. El frontend (HTML/CSS/JS) se comunica con un único punto de entrada `api.php` que actúa como API REST; PHP gestiona la lógica y la sesión, y MySQL almacena los datos.



Ciclo de una petición: el cliente envía una llamada AJAX con un parámetro `action` → la API responde en JSON

## 4.1 Capas del sistema

| CAPA                 | TECNOLOGÍA                        | FUNCIÓN                                            |
|----------------------|-----------------------------------|----------------------------------------------------|
| <b>Servidor</b>      | Apache (XAMPP)                    | Servir la aplicación en localhost / LAN            |
| <b>Backend</b>       | PHP 8                             | Lógica de negocio, sesiones, seguridad CSRF        |
| <b>Base de datos</b> | MySQL · phpMyAdmin                | Persistencia de usuarios, posts y contactos        |
| <b>API</b>           | PHP REST ( <code>api.php</code> ) | 24 acciones AJAX (posts, likes, admin, backups...) |
| <b>Frontend</b>      | HTML5 · CSS3 · JavaScript         | Vistas, maquetación responsive e interactividad    |
| <b>Despliegue</b>    | Scripts .bat · ngrok              | Arranque automático y túnel de acceso externo      |

## 4.2 Patrón arquitectónico

El proyecto sigue una variante del patrón **front controller** para los datos: todas las operaciones dinámicas pasan por `api.php`, que decide qué hacer según el parámetro `action` de la URL y el método HTTP. Las vistas ( `index.php`, `admin.php` ) son páginas PHP que sirven HTML, y la interactividad se delega al cliente con JavaScript, que consume la API.

## 4.3 Flujo de una petición AJAX

SCRIPT.JS – HELPER DE LLAMADAS A LA API

```
async function api(action, method = 'GET', body = null) {
  const opts = { method, headers: { 'Content-Type': 'application/json' } };
  if (body) opts.body = JSON.stringify(body);
  const res = await fetch(`api.php?action=${action}`, opts);
  return res.json(); // la API siempre responde JSON
}
```

El cliente nunca habla directamente con la base de datos: construye una petición, la API la valida y devuelve un objeto JSON con `success`, datos o `error`. Esto desacopla frontend y backend y centraliza la seguridad en un solo archivo.

## 4.4 Justificación del stack

### ¿Por qué PHP + MySQL?

Es el stack estándar de XAMPP, ampliamente soportado, ideal para aprender desarrollo de servidor y con excelente integración mediante `mysqli` y sentencias preparadas.

### ¿Por qué API REST propia?

Separar datos de presentación permite una UI fluida sin recargas, reutilizar la lógica desde varias vistas y razonar sobre la seguridad en un único punto de entrada.

### ¿Por qué vanilla JS?

Sin frameworks pesados: carga rápida, control total y demostración directa de DOM, `fetch` y eventos, que son la base de cualquier framework.

### ¿Por qué XAMPP?

Entorno local todo-en-uno (Apache + MySQL + PHP) que reproduce un servidor real y permite desplegar en clase con scripts `.bat` y ngrok.

### Decisión de diseño

Toda mutación de datos exige método **POST** y, cuando corresponde, rol admin; las lecturas usan **GET**. Esta separación hace el sistema predecible y más fácil de proteger.

# Modelo de datos

Base de datos `usuarios_db` (MySQL · InnoDB · utf8mb4) con tres tablas. Los posts se vinculan a su autor mediante clave foránea con borrado en cascada, garantizando la integridad referencial.

| 🗄️ usuarios    |              |
|----------------|--------------|
| id             | PK           |
| nombre         |              |
| username       | ÚNICO        |
| email          | ÚNICO        |
| password       | HASH         |
| favorite_game  |              |
| bio            |              |
| role           | ADMIN/MEMBER |
| fecha_registro |              |

| 🗄️ posts   |                  |
|------------|------------------|
| id         | PK               |
| title      |                  |
| content    |                  |
| category   |                  |
| author_id  | FK → USUARIOS.ID |
| likes      |                  |
| approved   |                  |
| created_at |                  |

relación 1 : N — un usuario puede tener muchos posts · `ON DELETE CASCADE`

| 🗄️ contacts |    |
|-------------|----|
| id          | PK |
| name        |    |
| email       |    |
| subject     |    |
| message     |    |
| read        |    |
| created_at  |    |



## Índices

Para optimizar consultas y ordenación se definen índices sobre `role`, `category`, `approved`, `created_at`, `likes` y `read`.

## 5.1 Tabla `usuarios`

Almacena las cuentas. El campo `password` guarda siempre un hash bcrypt, nunca la contraseña original. El `role` es un `ENUM` que limita los valores a `admin` o `member`.

| CAMPO                       | TIPO                | DESCRIPCIÓN                                        |
|-----------------------------|---------------------|----------------------------------------------------|
| <code>id</code>             | INT AUTO_INCREMENT  | Clave primaria.                                    |
| <code>nombre</code>         | VARCHAR(100)        | Nombre visible del usuario.                        |
| <code>username</code>       | VARCHAR(50) UNIQUE  | Nick único para mostrar e identificar.             |
| <code>email</code>          | VARCHAR(150) UNIQUE | Correo único, usado para iniciar sesión.           |
| <code>password</code>       | VARCHAR(255)        | Hash bcrypt de la contraseña.                      |
| <code>favorite_game</code>  | VARCHAR(100)        | Juego favorito (lista blanca de valores).          |
| <code>bio</code>            | TEXT                | Biografía corta del perfil.                        |
| <code>role</code>           | ENUM(admin,member)  | Rol del usuario, por defecto <code>member</code> . |
| <code>fecha_registro</code> | TIMESTAMP           | Fecha de alta automática.                          |

## 5.2 Tabla `posts`

| CAMPO                   | TIPO               | DESCRIPCIÓN                                 |
|-------------------------|--------------------|---------------------------------------------|
| <code>id</code>         | INT AUTO_INCREMENT | Clave primaria.                             |
| <code>title</code>      | VARCHAR(100)       | Título del hilo.                            |
| <code>content</code>    | TEXT               | Cuerpo del mensaje.                         |
| <code>category</code>   | VARCHAR(50)        | Categoría (fps, moba, hardware...).         |
| <code>author_id</code>  | INT · FK           | Autor → <code>usuarios.id</code> (CASCADE). |
| <code>likes</code>      | INT                | Contador de “me gusta”.                     |
| <code>approved</code>   | TINYINT(1)         | 1 publicado, 0 pendiente de moderación.     |
| <code>created_at</code> | TIMESTAMP          | Fecha de creación automática.               |

### 5.3 Tabla `contacts`

Recoge los mensajes del formulario de contacto. El campo reservado ``read`` se escribe siempre entre comillas invertidas por ser palabra reservada en SQL.

| CAMPO             | TIPO               | DESCRIPCIÓN                       |
|-------------------|--------------------|-----------------------------------|
| <b>id</b>         | INT AUTO_INCREMENT | Clave primaria.                   |
| <b>name</b>       | VARCHAR(100)       | Nombre de quien escribe.          |
| <b>email</b>      | VARCHAR(150)       | Correo de contacto.               |
| <b>subject</b>    | VARCHAR(200)       | Asunto del mensaje.               |
| <b>message</b>    | TEXT               | Texto del mensaje.                |
| <b>read</b>       | TINYINT(1)         | 1 leído por el admin, 0 sin leer. |
| <b>created_at</b> | TIMESTAMP          | Fecha de recepción.               |

### 5.4 Creación automática y datos de ejemplo

El archivo `db.php` crea la base de datos y las tablas si no existen, e inserta datos de ejemplo (*seeds*) en la primera visita: una cuenta de administrador, ocho miembros de demostración, quince hilos repartidos por categorías y cinco mensajes de contacto. Así el foro nunca aparece vacío.

DB.PHP – CREACIÓN DE LA TABLA POSTS (FRAGMENTO)

```
CREATE TABLE IF NOT EXISTS posts (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  title       VARCHAR(100) NOT NULL,  
  content     TEXT          NOT NULL,  
  category    VARCHAR(50)   NOT NULL DEFAULT 'general',  
  author_id   INT           NOT NULL,  
  likes       INT DEFAULT 0,  
  approved    TINYINT(1)   DEFAULT 0,  
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (author_id) REFERENCES usuarios(id) ON DELETE CASCADE,  
  INDEX idx_category (category), INDEX idx_likes (likes)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

#### Integridad referencial

Gracias a `ON DELETE CASCADE`, al eliminar un usuario se borran automáticamente todos sus posts, evitando registros huérfanos.

# Estructura de archivos

El código fuente se organiza por responsabilidad: vistas, procesadores de formularios, lógica compartida, API y recursos estáticos. Cada archivo tiene un cometido único y bien definido.

```
Proyecto-Intramodular/ └─ documentacio/ # memoria y Kanban + control de horas | └─  
NEXUS_Memoria.pdf | └─ NEXUS_Kanban_y_Horas.pdf └─ README.md # presentación del  
repositorio └─ gaming-foro-v2/ # código fuente de la aplicación └─ index.php # página  
principal del foro └─ api.php # API REST – todas las acciones AJAX └─ db.php # conexión +  
creación de tablas + seeds └─ bootstrap.php # sesión, usuario actual, CSRF, utilidades └─  
page_top.php # cabecera PHP compartida └─ admin.php # panel de administración └─  
login.php # pantalla de inicio de sesión └─ register.php # pantalla de registro └─  
logout.php # cierre de sesión y redirección └─ process_login.php # procesador de login └─  
process_register.php # procesador de registro └─ process_contact.php # procesador de  
contacto └─ styles.css # hoja de estilos completa (responsive) └─ script.js # JavaScript  
del frontend (AJAX, UI) └─ .htaccess # gzip, caché y cabeceras de seguridad └─  
ARRANCAR.bat # inicia XAMPP, red y cortafuegos └─ iniciar-tunel.bat # abre un túnel ngrok
```

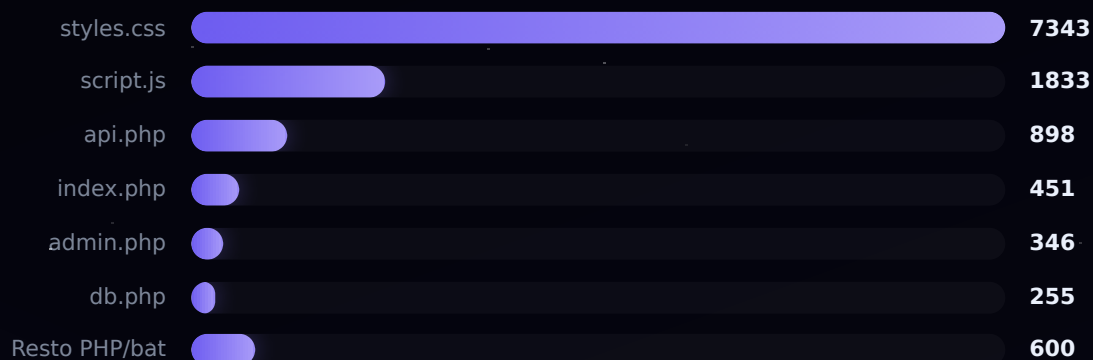


## 6.1 Descripción de cada archivo

| ARCHIVO                                 | DESCRIPCIÓN                                                                       |
|-----------------------------------------|-----------------------------------------------------------------------------------|
| <b>index.php</b>                        | Página principal: hero; listado del foro, miembros y contacto.                    |
| <b>api.php</b>                          | API REST con 24 acciones (lectura, mutación, admin y backups).                    |
| <b>db.php</b>                           | Conexión MySQL y creación automática de tablas y datos de ejemplo.                |
| <b>bootstrap.php</b>                    | Funciones compartidas: sesión, usuario actual, CSRF, utilidades.                  |
| <b>page_top.php</b>                     | Cabecera PHP que arranca la app y expone el usuario a JavaScript.                 |
| <b>admin.php</b>                        | Panel de administración (resumen, moderación, usuarios, backups, zona peligrosa). |
| <b>login.php</b><br><b>register.php</b> | Pantallas de inicio de sesión y registro.                                         |
| <b>process_*.php</b>                    | Procesadores de login, registro y contacto.                                       |
| <b>logout.php</b>                       | Destruye la sesión de forma segura y redirige.                                    |
| <b>styles.css</b>                       | Hoja de estilos completa de la estética Cosmos (responsive).                      |
| <b>script.js</b>                        | JavaScript del frontend: AJAX, render dinámico e interactividad.                  |
| <b>.htaccess</b>                        | Compresión gzip, caché del navegador y cabeceras de seguridad.                    |
| <b>ARRANCAR.bat</b>                     | Inicia XAMPP, configura la red y abre el cortafuegos.                             |
| <b>iniciar-tunel.bat</b>                | Abre un túnel ngrok para acceso por internet.                                     |

## 6.2 Volumen de código

El proyecto reúne en torno a **11.700 líneas**. El reparto refleja el peso del diseño visual y de la interactividad del cliente frente a la lógica de servidor, más compacta.



# Backend y API REST

El archivo `api.php` es el corazón del backend: un único punto de entrada que atiende las **24 acciones** del sistema. Cada acción se identifica por el parámetro `action` de la URL y exige un método HTTP concreto (GET para leer, POST para mutar).

## 7.1 Estructura del despachador

Al recibir una petición, la API fija las cabeceras de seguridad, lee la acción y el método, y recorre una cadena de bloques condicionales. Cuando uno coincide, ejecuta su lógica y termina con una respuesta JSON. Si ninguno coincide, devuelve un error 404 con la lista de acciones disponibles.

API.PHP – CABECERAS Y DESPACHO

```
header('Content-Type: application/json; charset=utf-8');
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');

$action = $_GET['action'] ?? '';
$method = $_SERVER['REQUEST_METHOD'];

if ($action === 'get_posts' && $method === 'GET') { /* ... */ }
if ($action === 'add_post' && $method === 'POST') { /* ... */ }
// ... 22 acciones más ...
```

## 7.2 Funciones auxiliares clave

| FUNCIÓN                                       | PROPÓSITO                                                              |
|-----------------------------------------------|------------------------------------------------------------------------|
| <b>isAdmin()</b> / <b>isLoggedIn()</b>        | Comprueban el rol y la sesión del usuario actual.                      |
| <b>requireAdmin()</b> / <b>requireLogin()</b> | Cortan la ejecución con error 403/401 si no se cumple.                 |
| <b>jsonOut()</b> / <b>jsonError()</b>         | Emiten respuestas JSON consistentes con código de estado.              |
| <b>postBody()</b>                             | Lee y decodifica el cuerpo JSON de las peticiones POST.                |
| <b>sanitizeText()</b>                         | Limpia texto ( <code>strip_tags</code> ) y aplica longitud máxima.     |
| <b>paginate()</b>                             | Pagina cualquier consulta y devuelve total, página y total de páginas. |

## 7.3 Catálogo de acciones — lectura (GET)

| ACCIÓN                 | ACCESO  | DESCRIPCIÓN                                                          |
|------------------------|---------|----------------------------------------------------------------------|
| <b>get_posts</b>       | Público | Feed del foro con filtro de categoría, búsqueda, orden y paginación. |
| <b>get_post</b>        | Público | Un hilo concreto por su ID.                                          |
| <b>get_users</b>       | Público | Lista de miembros con su recuento de posts y likes.                  |
| <b>get_user</b>        | Público | Perfil de un usuario concreto.                                       |
| <b>get_user_posts</b>  | Público | Hilos publicados por un usuario.                                     |
| <b>get_leaderboard</b> | Público | Ranking de miembros por likes acumulados.                            |
| <b>search</b>          | Público | Búsqueda de hilos por título y contenido.                            |
| <b>get_stats</b>       | Público | Estadísticas para la barra lateral y el hero.                        |
| <b>get_all_posts</b>   | Admin   | Todos los hilos, incluidos los pendientes de aprobación.             |
| <b>get_contacts</b>    | Admin   | Mensajes del formulario de contacto.                                 |
| <b>get_admin_stats</b> | Admin   | Métricas detalladas para el dashboard.                               |
| <b>backup_database</b> | Admin   | Descarga del volcado SQL completo.                                   |
| <b>backup_files</b>    | Admin   | Descarga de un ZIP con todos los archivos web.                       |
| <b>backup_full</b>     | Admin   | ZIP completo con archivos + volcado SQL embebido.                    |

## 7.4 Catálogo de acciones — mutación (POST)

| ACCIÓN                   | ACCESO      | DESCRIPCIÓN                                                    |
|--------------------------|-------------|----------------------------------------------------------------|
| <b>add_post</b>          | Login       | Crea un hilo (pendiente si es miembro, publicado si es admin). |
| <b>like_post</b>         | Público     | Incrementa el contador de likes de un hilo.                    |
| <b>approve_post</b>      | Admin       | Aprueba o rechaza un hilo pendiente.                           |
| <b>delete_post</b>       | Admin       | Elimina un hilo permanentemente.                               |
| <b>mark_contact_read</b> | Admin       | Marca un mensaje de contacto como leído.                       |
| <b>delete_contact</b>    | Admin       | Elimina un mensaje de contacto.                                |
| <b>update_user_role</b>  | Admin       | Cambia el rol de un usuario (admin / member).                  |
| <b>delete_user</b>       | Admin       | Elimina una cuenta de miembro y sus posts.                     |
| <b>reset_site</b>        | Admin + PIN | Borra datos según objetivo (posts, usuarios, todo, BBDD).      |
| <b>restore_files</b>     | Admin + PIN | Restaura los archivos web desde un ZIP subido.                 |

## 7.5 Ejemplo: obtener el feed del foro

La acción `get_posts` construye la consulta dinámicamente según los filtros recibidos, usando siempre sentencias preparadas. Solo devuelve hilos aprobados ( `approved = 1` ) y los une con su autor para mostrar el nick.

API.PHP – ACCIÓN GET\_POSTS (FRAGMENTO)

```
$sql = "SELECT p.id, p.title, p.content, p.category, p.likes,
           p.created_at, u.username, u.nombre
        FROM posts p JOIN usuarios u ON p.author_id = u.id
        WHERE p.approved = 1";

if ($category !== 'all') { $sql .= " AND p.category = ?"; }
if ($search !== '') { $sql .= " AND (p.title LIKE ? OR p.content LIKE ?)"; }

$sql .= $sort === 'popular'
    ? " ORDER BY p.likes DESC, p.created_at DESC"
    : " ORDER BY p.created_at DESC";

jsonOut( paginate($conn, $sql, $types, $params, $page, $perPage) );
```

## 7.6 Ejemplo: dar like en tiempo real

API.PHP – ACCIÓN LIKE\_POST

```
$stmt = $conn->prepare("UPDATE posts SET likes = likes + 1
                        WHERE id = ? AND approved = 1");
$stmt->bind_param('i', $postId);
$stmt->execute();

if ($stmt->affected_rows < 1) jsonError('Post no encontrado', 404);
jsonOut(['success' => true, 'likes' => $newLikes]);
```

El servidor devuelve el nuevo total de likes y el cliente actualiza el contador del botón al instante, sin recargar la página.

## 7.7 Formato de respuesta unificado

Toda la API responde con JSON y una estructura predecible. Esto simplifica el cliente: basta comprobar `success` o la presencia de `error`.

### Éxito

```
{
  "success": true,
  "likes": 42
}
```

### Error

```
{
  "error": "No autorizado",
  "success": false
}
```

## 7.8 Códigos de estado HTTP utilizados

| CÓDIGO                | SIGNIFICADO EN NEXUS                         |
|-----------------------|----------------------------------------------|
| 200 OK                | Operación correcta; datos en el cuerpo JSON. |
| 400 Bad Request       | Datos inválidos o parámetro incorrecto.      |
| 401 Unauthorized      | Acción que requiere iniciar sesión.          |
| 403 Forbidden         | Falta el rol admin o el PIN es incorrecto.   |
| 404 Not Found         | Recurso o acción inexistente.                |
| 429 Too Many Requests | Límite de envíos del formulario de contacto. |
| 500 / 503             | Error interno o base de datos no disponible. |

### Diseño defensivo

La API desactiva la salida de errores de PHP ( `display_errors = 0` ) y limpia el búfer antes de responder, de modo que un aviso interno nunca rompa el JSON que espera el cliente.

# Autenticación y seguridad

La seguridad se aborda en varias capas: cifrado de contraseñas, sentencias preparadas, validación de entrada, gestión robusta de sesiones, protección CSRF y límites contra abuso. Las acciones más peligrosas se protegen además con un PIN.

## 8.1 Cifrado de contraseñas

Las contraseñas nunca se guardan en texto plano. Se cifran con **bcrypt** mediante `password_hash()` (coste 12) y se verifican con `password_verify()`. Aunque alguien accediera a la base de datos, no podría leer las contraseñas originales.

PROCESS\_REGISTER.PHP – CIFRADO AL REGISTRAR

```
$passwordHash = password_hash($password, PASSWORD_BCRYPT, ['cost' => 12]);  
// ... se almacena $passwordHash, jamás $password
```

## 8.2 Protección contra inyección SQL

Todas las consultas con datos del usuario usan **sentencias preparadas** con `bind_param`. Los valores viajan separados de la instrucción SQL, por lo que no pueden alterar su estructura.

PATRÓN USADO EN TODO EL PROYECTO

```
$stmt = $conn->prepare('SELECT id, password, role FROM usuarios WHERE email = ?');  
$stmt->bind_param('s', $email);  
$stmt->execute();
```

## 8.3 Validación de entrada

- › El **email** se valida con `FILTER_VALIDATE_EMAIL` y se normaliza a minúsculas.
- › El **nick** solo admite letras, números, guiones y guiones bajos (3-18 caracteres).
- › Los **textos** se limpian con `strip_tags` y se truncan a su longitud máxima.
- › La **categoría** y el **juego favorito** se validan contra una lista blanca.

## 8.4 Gestión de sesiones

Tras un login correcto se llama a `session_regenerate_id(true)` para prevenir el secuestro de sesión (*session fixation*). El rol guardado en la sesión se vuelve a sincronizar con la base de datos en cada carga, de modo que un cambio de rol surte efecto de inmediato. El logout destruye la sesión y borra la cookie correctamente.

PROCESS\_LOGIN.PHP – SESIÓN SEGURA

```
session_regenerate_id(true); // nuevo ID al autenticar
$_SESSION['user_id'] = (int) $user['id'];
$_SESSION['role'] = $user['role'];
$_SESSION['username'] = $user['username'];
```

## 8.5 Defensa contra fuerza bruta

El procesador de login cuenta los intentos fallidos por sesión. Tras **10 intentos**, bloquea el acceso durante **10 minutos** y avisa al usuario de cuántos intentos le quedan.

## 8.6 Protección CSRF y honeypot

Se genera un token CSRF por sesión ( `csrf_token()` / `csrf_verify()` ) y se comprueba el origen de las peticiones (*Referer* / *Host*). El formulario de contacto incluye además un campo trampa oculto (*honeypot*): si un bot lo rellena, la petición se rechaza.

BOOTSTRAP.PHP – TOKEN CSRF

```
function csrf_token(): string {
    if (empty($_SESSION['csrf_token']))
        $_SESSION['csrf_token'] = bin2hex(random_bytes(24));
    return $_SESSION['csrf_token'];
}
```

## 8.7 PIN para acciones destructivas

Las operaciones irreversibles —vaciar tablas, reset total o eliminar la base de datos, y restaurar el sitio desde un ZIP— exigen, además del rol admin, un **PIN de seguridad**. Es una segunda barrera frente a errores accidentales o a una sesión admin abierta por descuido.

```
API.PHP – VERIFICACIÓN DE PIN
```

```
if ($pin !== '565656') jsonResponse('PIN incorrecto', 403);

$allowed = ['posts', 'users', 'contacts', 'all', 'database'];
if (!in_array($target, $allowed, true)) jsonResponse('Objetivo no válido');
```

## 8.8 Salvaguardas adicionales del admin

- Un administrador **no puede cambiar su propio rol** ni eliminarse a sí mismo.
- Un administrador **no puede eliminar a otro administrador**.
- La restauración bloquea rutas con `..` para impedir *path traversal*.
- El restaurador solo escribe en rutas conocidas ( `gaming-foro-v2/` e `ip.php` ).

## 8.9 Cabeceras de seguridad

Tanto la API como el `.htaccess` añaden cabeceras defensivas:

| CABECERA                               | EFFECTO                                              |
|----------------------------------------|------------------------------------------------------|
| <b>X-Content-Type-Options: nosniff</b> | Evita que el navegador adivine el tipo de contenido. |
| <b>X-Frame-Options</b>                 | Impide incrustar el sitio en iframes (clickjacking). |
| <b>X-XSS-Protection</b>                | Activa el filtro anti-XSS del navegador.             |
| <b>Referrer-Policy</b>                 | Limita la información de origen enviada a terceros.  |
| <b>Cache-Control: no-store</b>         | Impide cachear respuestas dinámicas sensibles.       |

### Resumen de seguridad

Bcrypt + sentencias preparadas + validación + sesiones regeneradas + CSRF + límites de intentos + PIN forman una defensa en profundidad razonable para el alcance del proyecto.



# Funcionalidades principales

NEXUS reúne las funcionalidades esperadas de un foro moderno de comunidad: autenticación, publicación y consumo de contenido, interacción social y herramientas de comunidad, todo con una experiencia fluida basada en AJAX.



## Autenticación de usuarios

Registro con validaciones, inicio de sesión con gestión de sesión PHP, cifrado de contraseñas con hash y mensajes de error claros.



## Sistema de foros

Creación y listado de hilos por categoría (FPS, MOBA, estrategia, noticias, hardware), publicación de mensajes, paginación y orden por recientes o populares.



## Sistema de likes

Dar y quitar “me gusta” mediante AJAX, con contador dinámico actualizado en tiempo real sin recargar la página.



## Panel de administración

Dashboard con métricas, moderación de hilos y mensajes, gestión de usuarios (roles y baneos) y estadísticas del foro.



## Copias de seguridad

Descarga de backup SQL, ZIP de los archivos del proyecto o backup completo, además de restauración de la web desde un ZIP previo.



## Contacto y comunidad

Formulario de contacto, ranking de miembros (leaderboard) y perfiles con juego favorito y biografía.

## Roles y zona de seguridad

Dos roles de usuario: **administrador** y **miembro**. Las acciones destructivas (borrado de posts, usuarios o mensajes, restauración o reinicio del sitio) están protegidas por un **PIN de seguridad** adicional.

## 9.1 Foro: filtrado, búsqueda y orden

El listado de hilos se puede acotar por categoría con la barra de filtros, buscar por texto y ordenar por *recientes* o *populares*. La búsqueda usa *debounce* para no lanzar una petición por cada tecla, y la paginación trae los hilos en lotes para mantener la carga ligera.

### FILTRAR

#### Por categoría

Botones de categoría que recargan el feed mostrando solo los hilos de ese tema.

### BUSCAR

#### Texto libre

Coincidencias en título y contenido, con un mínimo de 2 caracteres.

### ORDENAR

#### Recientes / populares

Por fecha de creación o por número de likes acumulados.

## 9.2 Publicación de hilos

El compositor permite a un miembro escribir un título, un cuerpo y elegir categoría. El cliente valida longitudes mínimas (título  $\geq 5$ , contenido  $\geq 10$ ) antes de enviar, y muestra un contador de caracteres en vivo. El servidor vuelve a validar y sanear: la validación de cliente mejora la experiencia, pero la de servidor es la que protege.

## 9.3 Comunidad: perfiles y ranking

- › **Perfiles:** cada miembro tiene nombre, nick, juego favorito, biografía, recuento de posts y likes totales.
- › **Leaderboard:** ranking de los miembros más valorados, calculado por la suma de likes de sus hilos aprobados.
- › **Estadísticas públicas:** el hero y la barra lateral muestran totales de hilos, miembros y likes.

### Gamificación sutil

El ranking convierte la calidad de las aportaciones en reputación visible, incentivando publicaciones útiles sin necesidad de mecánicas complejas.

## 9.4 Formulario de contacto

Cualquier visitante puede escribir al equipo. El procesador `process_contact.php` valida todos los campos, aplica un límite de un envío por minuto y un honeypot anti-bot, y guarda el mensaje en la tabla `contacts` para que el administrador lo lea desde su panel.

| VALIDACIÓN | REGLA                                  |
|------------|----------------------------------------|
| Nombre     | Entre 2 y 80 caracteres.               |
| Asunto     | Entre 4 y 180 caracteres.              |
| Mensaje    | Entre 10 y 2000 caracteres.            |
| Email      | Formato válido, máximo 150 caracteres. |
| Frecuencia | Un envío cada 60 segundos por sesión.  |

## 9.5 Mapa de funcionalidades por actor

| FUNCIONALIDAD                | VISITANTE | MIEMBRO       | ADMIN        |
|------------------------------|-----------|---------------|--------------|
| Leer hilos y perfiles        | Sí        | Sí            | Sí           |
| Buscar y filtrar             | Sí        | Sí            | Sí           |
| Dar likes                    | Sí        | Sí            | Sí           |
| Enviar contacto              | Sí        | Sí            | Sí           |
| Publicar hilos               | —         | Sí (moderado) | Sí (directo) |
| Moderar y gestionar usuarios | —         | —             | Sí           |
| Backups y restauración       | —         | —             | Sí           |

# Panel de administración

El panel ( `admin.php` ) es una aplicación de una sola página con cinco pestañas. Solo es accesible para administradores; un visitante o miembro ve una pantalla de acceso restringido.

## 10.1 Las cinco pestañas

### PESTAÑA 1

#### Resumen

Dashboard con métricas globales: usuarios, hilos aprobados y pendientes, mensajes sin leer y likes totales, además de categorías y top de hilos.

### PESTAÑA 2

#### Moderación

Lista de todos los hilos (incluidos los pendientes) para aprobar, rechazar o eliminar, y gestión de los mensajes de contacto.

### PESTAÑA 3

#### Usuarios

Búsqueda y filtrado de miembros, cambio de rol y eliminación de cuentas con sus salvaguardas.

### PESTAÑA 4

#### Backups

Descarga de SQL, ZIP de archivos o backup completo, y restauración del sitio desde un ZIP.

### PESTAÑA 5

#### Zona peligrosa

Acciones irreversibles protegidas por PIN: vaciar tablas, reset total del foro o eliminar la base de datos completa. Se desbloquean solo tras verificar el PIN.

## 10.2 Control de acceso del panel

El cliente comprueba el rol del usuario expuesto en `window.NEXUS_CURRENT_USER` y, si no es admin, muestra el bloque de acceso denegado y oculta el contenido. La protección real, sin embargo, está en el servidor: cada acción admin de la API exige `requireAdmin()`, de modo que manipular el cliente no concede privilegios.

### 10.3 Pestaña Resumen — métricas

La acción `get_admin_stats` alimenta las tarjetas del dashboard con cifras en vivo y dos desgloses: hilos y likes por categoría, y los cinco hilos más populares.



### 10.4 Pestaña Moderación

Muestra todos los hilos ordenados con los pendientes primero. Cada fila ofrece botones para **aprobar**, **rechazar** o **eliminar**. Los cambios se hacen por AJAX y la lista se refresca sin recargar.

API.PHP – APROBAR / RECHAZAR UN HILO

```
if ($action === 'approve_post' && $method === 'POST') {  
    requireAdmin();  
    $stmt = $conn->prepare("UPDATE posts SET approved = ? WHERE id = ?");  
    $stmt->bind_param('ii', $approved, $postId);  
    $stmt->execute();  
    jsonOut(['success' => true, 'approved' => (bool) $approved]);  
}
```

## 10.5 Pestaña Usuarios

El administrador puede buscar por nombre o nick, filtrar por rol, promover o degradar usuarios y eliminar cuentas. La tabla se cachea en el cliente para filtrar al instante sin volver a pedir datos.

- › **Cambiar rol:** `update_user_role` alterna entre admin y member.
- › **Eliminar usuario:** `delete_user` borra la cuenta y, por CASCADE, sus hilos.
- › **Salvaviduas:** no puede tocarse a uno mismo ni eliminar a otro admin.

## 10.6 Gestión de mensajes de contacto

Desde Moderación, el administrador ve los mensajes recibidos, los marca como leídos ( `mark_contact_read` ) o los elimina ( `delete_contact` ). Los no leídos se destacan y se contabilizan en el resumen.

### Experiencia del moderador

Todas las acciones del panel devuelven un *toast* de confirmación o error, de modo que el administrador siempre sabe si la operación tuvo éxito.

## 10.7 Pestaña Zona peligrosa

Concentra las operaciones irreversibles, deliberadamente separadas del resto. El bloque permanece bloqueado hasta introducir y verificar el PIN; solo entonces se muestran las tarjetas de acción, cada una con una advertencia clara de su efecto.

| ACCIÓN                       | EFFECTO                                                                  |
|------------------------------|--------------------------------------------------------------------------|
| Eliminar todos los posts     | Borra todos los hilos; los usuarios permanecen.                          |
| Eliminar usuarios (no admin) | Borra miembros y sus posts; conserva la cuenta admin.                    |
| Vaciar mensajes de contacto  | Elimina todos los mensajes recibidos.                                    |
| Reset total del foro         | Borra posts, miembros y mensajes; queda solo el admin.                   |
| Eliminar base de datos       | Elimina <code>usuarios_db</code> ; se recrea vacía en la próxima visita. |

### Diseño consciente del riesgo

El aspecto visual escala con el peligro: la acción más destructiva (eliminar la BBDD) usa un botón “nuclear” diferenciado. Combinado con el PIN y la confirmación, reduce al mínimo el riesgo de un borrado accidental.

## 10.8 Resumen del panel

El panel concentra en una sola página toda la operativa de gestión: observar el estado del foro, moderar contenido, administrar la comunidad, asegurar los datos y, en último término, reiniciar el sistema de forma controlada.

# Copias de seguridad y restauración

NEXUS ofrece un sistema de copias de seguridad completo desde el propio panel, sin depender de herramientas externas, además de la opción clásica de phpMyAdmin. Incluye también la restauración del sitio desde un ZIP previo.

## 11.1 Tipos de copia



### Backup SQL

`backup_database` genera un volcado `.sql` con la estructura y los datos de las tres tablas, listo para reimportar.



### ZIP de archivos

`backup_files` empaqueta todos los archivos web del proyecto en un `.zip`.



### Backup completo

`backup_full` crea un único ZIP con los archivos web y el volcado SQL embebido dentro.

## 11.2 Cómo se genera el volcado SQL

El servidor recorre las tablas con `SHOW TABLES`, obtiene su definición con `SHOW CREATE TABLE` y vuelca los registros como sentencias `INSERT`, escapando cada valor. El resultado es un script reproducible que recrea la base de datos desde cero.

API.PHP – GENERACIÓN DEL VOLCADO (FRAGMENTO)

```
$tables = $conn->query("SHOW TABLES")->fetch_all(MYSQLI_NUM);
foreach ($tables as [$table]) {
    $create = $conn->query("SHOW CREATE TABLE `{$table}`");
    $out .= "DROP TABLE IF EXISTS `{$table}`;\n" . $createSql . ";\n";
    // ... vuelca cada fila como INSERT escapado ...
}
```



### 11.3 Restauración desde un ZIP

La acción `restore_files` recibe un ZIP por *multipart/form-data*, exige rol admin y PIN, y reescribe los archivos del proyecto. Es la operación más delicada del sistema, por lo que incorpora varias comprobaciones de seguridad.

- › Comprueba que el archivo sea realmente un `.zip` válido.
- › Rechaza cualquier entrada que contenga `..` (protección contra *path traversal*).
- › Solo restaura rutas conocidas: `gaming-foro-v2/` e `ip.php`.
- › Crea los directorios intermedios necesarios y reporta cuántos archivos se restauraron u omitieron.

### 11.4 Flujo de copia y restauración



#### Autonomía de mantenimiento

El administrador puede asegurar y recuperar el sitio íntegro sin tocar el sistema de archivos ni la consola: todo desde la interfaz, con la red de seguridad del PIN.

# Frontend: HTML, CSS y JavaScript

El frontend combina vistas PHP que generan HTML semántico, una hoja de estilos extensa de estética **Cosmos** (más de 7.300 líneas) y un JavaScript modular sin frameworks que aporta toda la interactividad.

## 12.1 Sistema de diseño — tokens

La identidad visual se construye sobre variables CSS ( `:root` ) que definen una paleta de fondos “void”, bordes translúcidos, jerarquía de texto y un acento de starlight. Centralizar los valores permite mantener la coherencia y ajustar el tema en un solo lugar.

STYLES.CSS — TOKENS DE DISEÑO (FRAGMENTO)

```
:root {
  /* Fondos — capas del vacío */
  --void: #020208;  --surface: #0a0a12;
  --surface-2: #10101a;  --surface-3: #181826;
  /* Texto */
  --white: #f0f0f8;    --white-dim: rgba(240,240,248,.55);
  /* Bordes y acento */
  --border: rgba(255,255,255,.06);
  --accent-bright: #ffffff;
}
```

## 12.2 Tipografía y estética

Se combinan dos familias de Google Fonts: **Cormorant Garamond** (serif de display, para titulares con personalidad) y **DM Sans** (sans limpia para la interfaz). El conjunto evoca un “observatorio”: luz fría sobre negro profundo, con un fondo de estrellas animado por canvas.

## 12.3 Organización de la hoja de estilos

El CSS está dividido en secciones rotuladas: header, botones, hero, foro, tarjetas de post, sidebar, contacto, footer, páginas de auth, panel admin, toasts, animaciones, efectos decorativos, responsive, reduced-motion, alto contraste y print. Esta organización facilita localizar y mantener cada bloque.

## 12.4 Arquitectura del JavaScript

El archivo `script.js` (más de 1.800 líneas) se organiza en funciones con responsabilidad única, agrupadas por área. Se ejecuta tras cargar el DOM y detecta qué página está activa mediante el atributo `data-page` para inicializar solo lo necesario.

| ÁREA            | FUNCIONES REPRESENTATIVAS                                                                                                             |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Fondo galáctico | <code>initGalaxy</code> , <code>buildStars</code> , <code>spawnShooter</code> , <code>drawFrame</code>                                |
| Núcleo / API    | <code>api</code> , <code>toast</code> , <code>esc</code> , <code>fmtNumber</code> , <code>fmtDate</code>                              |
| Foro            | <code>renderPosts</code> , <code>likePost</code> , <code>setupComposer</code> , <code>setupFilters</code>                             |
| Comunidad       | <code>renderMembersList</code> , <code>renderLeaderboardExtended</code>                                                               |
| Contacto        | <code>setupContactForm</code>                                                                                                         |
| Admin           | <code>renderAdminPageFull</code> , <code>initAdminTabs</code> , <code>renderAdminUsersTable</code> , <code>initAdminBackupsTab</code> |
| UX              | <code>setupScrollReveal</code> , <code>setupBackToTop</code> , <code>setupStickyHeader</code> , <code>setupKeyboardShortcuts</code>   |

## 12.5 Render dinámico sin recargas

El listado del foro se construye en el cliente a partir del JSON de la API. Cada hilo se transforma en una tarjeta HTML, escapando el contenido para evitar XSS, y se enganchan los eventos de like y de abrir el compositor.

SCRIPT.JS – PATRÓN DE RENDER (ESQUEMA)

```
async function renderPosts() {
  const data = await api('get_posts' + query);
  container.innerHTML = data.data.map(p => cardTemplate(p)).join('');
  // re-enganchar listeners de like y composer
}
```

## 12.6 El fondo de estrellas (canvas)

El elemento `<canvas id="starfield">` dibuja un cielo estrellado vivo: capas de niebla, estrellas con parpadeo, estrellas fugaces ocasionales y un sutil rastro de polvo que reacciona al ratón. El bucle de animación se detiene cuando la pestaña no es visible para ahorrar recursos.

- › **buildFog / buildStars:** generan las capas de fondo según el tamaño de pantalla.
- › **spawnShooter / spawnDust:** añaden estrellas fugaces y partículas reactivas.
- › **visibilitychange:** pausa el render al ocultar la pestaña.

## 12.7 Componentes de interfaz reutilizables

### Toasts

Notificaciones temporales de éxito, error o info que confirman cada acción del usuario.

### Skeletons

Estados de carga que reservan el espacio del contenido mientras llega la respuesta de la API.

### Avatares generados

Iniciales con color derivado del nick, sin necesidad de subir imágenes.

### Contadores de caracteres

Feedback en vivo de la longitud en los formularios de publicación y contacto.

## 12.8 Persistencia de preferencias

El cliente recuerda preferencias de navegación del usuario —como la categoría y el orden elegidos— para restaurarlas en visitas siguientes, mejorando la continuidad de la experiencia sin intervención del servidor.

## 12.9 Cabecera compartida y datos al cliente

El archivo `page_top.php` arranca la aplicación y prepara un objeto JSON del usuario actual —sin la contraseña— que se inyecta en `window.NEXUS_CURRENT_USER`. Así el JavaScript sabe quién ha iniciado sesión y adapta la interfaz (botones de sesión, acceso al panel) en el cliente.

```
PAGE_TOP.PHP – EXPONER EL USUARIO A JS

$userForJs = [
    'id' => (int) $currentUser['id'],
    'username' => $currentUser['username'],
    'role' => $currentUser['role'],
    // nunca se incluye el hash de la contraseña
];
```

### Separación de responsabilidades

PHP entrega la página y el estado inicial; JavaScript se ocupa de la interactividad y consume la API. Cada capa hace una cosa y la hace bien.

# Experiencia de usuario y accesibilidad

Más allá de la funcionalidad, NEXUS cuida cómo se siente: una interfaz responsive, rápida y respetuosa con las preferencias del usuario, con detalles que facilitan la navegación.

## 13.1 Diseño responsive

La hoja de estilos define varios *breakpoints* para adaptar la maquetación a móvil, tablet y escritorio. En pantallas pequeñas, la navegación principal se colapsa en un menú hamburguesa y las rejillas de tarjetas pasan a una sola columna.

## 13.2 Atajos de teclado

| ATAJO | ACCIÓN                                     |
|-------|--------------------------------------------|
| /     | Enfoca el buscador del foro.               |
| N     | Abre el compositor de nuevo hilo.          |
| Esc   | Cierra diálogos y menús abiertos.          |
| Enter | Lanza la búsqueda desde el campo de texto. |

## 13.3 Detalles de navegación

- › **Header pegajoso:** la cabecera se compacta al hacer scroll para ganar espacio.
- › **Botón “volver arriba”:** aparece tras desplazarse y devuelve al inicio suavemente.
- › **Scroll suave:** los enlaces de ancla animan el desplazamiento entre secciones.
- › **Scroll reveal:** los bloques aparecen con una animación sutil al entrar en pantalla.

## 13.4 Accesibilidad y preferencias

### Reduced motion

Con `prefers-reduced-motion` se desactivan las animaciones para usuarios sensibles al movimiento.

### Alto contraste

Estilos específicos para `prefers-contrast` que refuerzan la legibilidad.

### HTML semántico

Uso de `header`, `nav`, `main`, `section` y atributos `aria-*` en elementos decorativos.

### Print styles

Reglas dedicadas para una impresión limpia de las páginas.

## 13.5 Rendimiento percibido

La combinación de carga por AJAX, esqueletos de carga y actualizaciones puntuales (en vez de recargas completas) hace que la aplicación se sienta rápida. El `.htaccess` refuerza esto con compresión gzip y caché agresiva de los recursos estáticos.



### Equilibrio

Los recursos que no cambian se cachean al máximo; los dinámicos nunca se cachean. Así la web carga rápido sin servir datos obsoletos.

# Despliegue y puesta en marcha

El proyecto se despliega sobre XAMPP en Windows con dos scripts que automatizan el arranque y el acceso externo. La base de datos se crea sola en la primera visita, sin pasos manuales.

## 14.1 Requisitos

- › **XAMPP** para Windows con Apache y MySQL.
- › **PHP 8.0** o superior (incluido en XAMPP).
- › **ZipArchive** habilitado (activo por defecto en XAMPP), necesario para los backups.

## 14.2 Instalación en cuatro pasos

| PASO | ACCIÓN                                                                                               |
|------|------------------------------------------------------------------------------------------------------|
| 1    | Copiar la carpeta <code>gaming-foro-v2/</code> dentro de <code>C:\xampp\htdocs\</code> .             |
| 2    | Ejecutar <code>ARRANCAR.bat</code> como administrador (inicia Apache + MySQL y abre el cortafuegos). |
| 3    | Abrir <code>http://localhost/gaming-foro-v2/</code> en el navegador.                                 |
| 4    | La base de datos y las tablas se crean automáticamente en la primera visita.                         |

## 14.3 El script ARRANCAR.bat

Este script automatiza todo el arranque: comprueba permisos de administrador, inicia los servicios de Apache y MySQL si no están corriendo, configura la red local y abre el puerto 80 en el cortafuegos, y por último detecta y muestra la IP del equipo para compartir el foro en clase.



## 14.4 Qué hace ARRANCAR.bat paso a paso

- **Permisos:** verifica que se ejecuta como administrador; si no, avisa y se detiene.
- **Servicios:** arranca Apache y MySQL comprobando antes si ya están en marcha.
- **Red privada:** pone los adaptadores activos en perfil de red privada con PowerShell.
- **Cortafuegos:** recrea reglas para abrir el puerto 80 en todos los perfiles.
- **IP:** detecta la IP con prioridad (hotspot > 192.168.x > 10.x) y la muestra en pantalla.

ARRANCAR.BAT – APERTURA DEL PUERTO 80 (FRAGMENTO)

```
REM Abrir puerto 80 en el Firewall de Windows
netsh advfirewall firewall add rule name="NEXUS-HTTP-80" ^
    dir=in action=allow protocol=TCP localport=80 profile=any enable=yes

REM Mostrar la URL de acceso desde la red
echo http://!MYIP!/gaming-foro-v2/
```

## 14.5 Acceso desde otros dispositivos



### Red local (LAN)

**ARRANCAR.bat** muestra la IP del equipo para compartir `http://<IP>/gaming-foro-v2/` con dispositivos del mismo WiFi.



### Por internet

Con **iniciar-tunel.bat** se abre un túnel **ngrok** que genera una URL pública para acceder desde fuera de la red.

## 14.6 El túnel ngrok

El script `iniciar-tunnel.bat` comprueba que Apache esté escuchando en el puerto 80 y, si es así, levanta un túnel ngrok hacia ese puerto. ngrok devuelve una URL pública temporal que permite que cualquiera, dentro o fuera de la red del centro, acceda al foro.

```
INICIAR-TUNEL.BAT - ARRANQUE DEL TÚNEL

REM Comprobar que Apache escucha en el puerto 80
netstat -an | find ":80 " | find "LISTENING"

REM Arrancar el túnel ngrok hacia el puerto 80
ngrok http 80 --log=stdout
```

## 14.7 Mantenimiento de la base de datos

Desde **Admin → Backups** se puede descargar el SQL, un ZIP de la web o un backup completo, así como restaurar la web desde un ZIP previo. La exportación e importación SQL también está disponible a través de phpMyAdmin para quien prefiera la vía clásica.

### Despliegue pensado para el aula

Toda la cadena —arrancar servicios, abrir firewall, detectar IP y compartir por LAN o ngrok— está automatizada para presentar el proyecto en clase con un par de clics.

# Pruebas y validación

Se dedicaron 2,5 horas a pruebas funcionales y corrección de bugs. La validación se centró en verificar que cada requisito funcional se cumpliera y que los controles de seguridad respondieran ante entradas incorrectas o malintencionadas.

## 15.1 Casos de prueba funcionales

| CASO                  | ENTRADA              | RESULTADO ESPERADO                |
|-----------------------|----------------------|-----------------------------------|
| Registro válido       | Datos correctos      | Cuenta creada y sesión iniciada.  |
| Email duplicado       | Email ya existente   | Error claro, sin crear cuenta.    |
| Login correcto        | Credenciales válidas | Acceso; admin redirige al panel.  |
| Login fallido x10     | Contraseña errónea   | Bloqueo temporal de 10 minutos.   |
| Publicar como miembro | Hilo válido          | Hilo en estado pendiente.         |
| Publicar como admin   | Hilo válido          | Hilo publicado directamente.      |
| Dar like              | Click en el botón    | Contador +1 sin recargar.         |
| Buscar                | Término existente    | Resultados filtrados.             |
| Contacto              | Mensaje válido       | Guardado y confirmación.          |
| Contacto x2 en 1 min  | Segundo envío        | Rechazo por límite de frecuencia. |

## 15.2 Pruebas de seguridad y permisos

| CASO                                   | RESULTADO ESPERADO                          |
|----------------------------------------|---------------------------------------------|
| Acción admin sin sesión admin          | Error 403, sin efecto.                      |
| Reset con PIN incorrecto               | Error 403, sin borrado.                     |
| Admin intenta autoeliminarse           | Operación bloqueada.                        |
| Restaurar ZIP con ruta <code>..</code> | Entrada omitida (path traversal).           |
| Entrada con etiquetas HTML             | Texto saneado con <code>strip_tags</code> . |
| Categoría no permitida                 | Se sustituye por <code>general</code> .     |

## 15.3 Bugs detectados y corregidos

- › Avisos de PHP que rompían el JSON de la API → se silenció la salida de errores y se limpió el búfer.
- › Descargas de backup corruptas por compresión → se desactivó gzip en esas respuestas.
- › Palabra reservada `read` en SQL → se escapó con comillas invertidas.
- › Sesión apuntando a un usuario eliminado → se limpia la sesión al no encontrarlo.

## 15.4 Validación cruzada cliente/servidor

Se comprobó que las validaciones del cliente (longitudes, formato) sean siempre repetidas en el servidor, de modo que saltarse el JavaScript no permita guardar datos inválidos. La validación de cliente es comodidad; la de servidor es la garantía.

### Resultado

Todos los casos de prueba se superaron tras la fase de depuración, dejando el sistema estable y predecible en sus rutas principales.

# Conclusiones y mejoras futuras

NEXUS cumple su objetivo: una aplicación web full-stack completa, segura y mantenible que cubre el ciclo de vida del software de principio a fin, con una identidad visual cuidada y un despliegue pensado para el aula.

## 16.1 Objetivos alcanzados

- › Autenticación segura con bcrypt, sesiones robustas y defensa contra fuerza bruta. ✓
- › Sistema de foros completo con categorías, búsqueda, orden y paginación. ✓
- › Likes en tiempo real vía AJAX. ✓
- › Panel de administración con moderación, gestión de usuarios y estadísticas. ✓
- › Copias de seguridad SQL/ZIP y restauración protegida por PIN. ✓
- › Despliegue automatizado en LAN e internet. ✓

## 16.2 Aprendizajes

### Técnicos

Diseño de API REST, sentencias preparadas, gestión de sesiones, manejo de ZIP y volcados SQL, y arquitectura cliente-servidor real.

### Metodológicos

Planificar con Kanban, registrar horas, dividir el trabajo en módulos cerrados y documentar de forma profesional.

## 16.3 Líneas de mejora futura

| MEJORA                                  | BENEFICIO                                            |
|-----------------------------------------|------------------------------------------------------|
| Likes por usuario (tabla intermedia)    | Evitar likes repetidos y poder retirarlos.           |
| Respuestas / comentarios en hilos       | Conversaciones anidadas, no solo hilos sueltos.      |
| Edición de perfil y avatar real         | Mayor personalización de la comunidad.               |
| Token CSRF en todas las peticiones AJAX | Reforzar la protección más allá del origen.          |
| Notificaciones por email                | Avisar de aprobaciones y respuestas.                 |
| Migrar a HTTPS y a un hosting real      | Seguridad de transporte y disponibilidad permanente. |
| Paginación infinita en el feed          | Navegación más fluida en listados largos.            |

## 16.4 Valoración final

El proyecto demuestra el dominio del desarrollo web full-stack: desde el modelado de datos y la lógica de servidor hasta el diseño de interfaz, la seguridad y el despliegue. Las 48,5 horas invertidas se traducen en una aplicación funcional, coherente y bien documentada, con una base sólida sobre la que seguir creciendo.

### Cierre

NEXUS no es solo un ejercicio académico: es un foro real, usable y desplegable, que aplica buenas prácticas de seguridad y experiencia de usuario en un producto con identidad propia.

# Referencia rápida

Credenciales, comandos y datos útiles para ejecutar y evaluar el proyecto.

## A.1 Credenciales de acceso

| ROL              | ACCESO                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------|
| Administrador    | Email <code>admin@nexusboard.gg</code> · Contraseña <code>Admin123!</code> · Nick <code>nexusadmin</code> |
| PIN de seguridad | <code>565656</code> (zona peligrosa y restauración)                                                       |
| Usuarios demo    | ProSniper88, LunarGG, GalaxyBuilder, MidLaneKing, RifleQueen, FortBuilder, TacticalAna, HardwareGuru      |
| Contraseña demo  | <code>Demo1234!</code> (para todos los usuarios de demostración)                                          |

## A.2 Datos de ejemplo precargados

|          |       |          |            |
|----------|-------|----------|------------|
| 9        | 15    | 5        | 6          |
| USUARIOS | HILOS | MENSAJES | CATEGORÍAS |

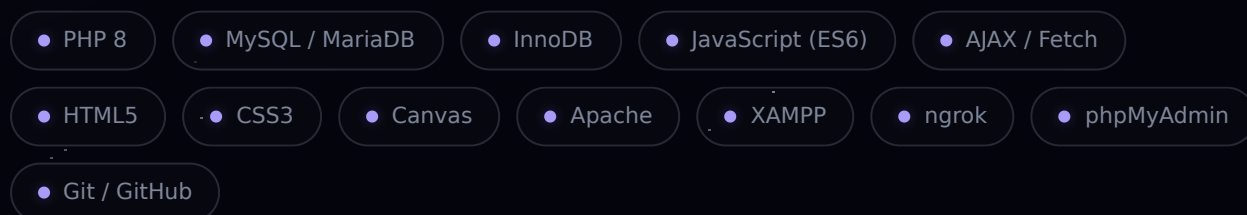
## A.3 URLs del proyecto

| RECURSO      | URL                                                    |
|--------------|--------------------------------------------------------|
| Foro (local) | <code>http://localhost/gaming-foro-v2/</code>          |
| Panel admin  | <code>http://localhost/gaming-foro-v2/admin.php</code> |
| Acceso LAN   | <code>http://&lt;IP&gt;/gaming-foro-v2/</code>         |
| phpMyAdmin   | <code>http://localhost/phpmyadmin/</code>              |

## A.4 Glosario técnico

| TÉRMINO                    | DEFINICIÓN                                                                             |
|----------------------------|----------------------------------------------------------------------------------------|
| <b>AJAX</b>                | Técnica para intercambiar datos con el servidor sin recargar la página.                |
| <b>API REST</b>            | Interfaz que expone operaciones sobre recursos mediante HTTP y JSON.                   |
| <b>bcrypt</b>              | Algoritmo de hash de contraseñas resistente a ataques por fuerza bruta.                |
| <b>CSRF</b>                | Ataque que fuerza acciones no deseadas; se mitiga con tokens y verificación de origen. |
| <b>Sentencia preparada</b> | Consulta SQL con parámetros separados que evita la inyección.                          |
| <b>InnoDB</b>              | Motor de MySQL con soporte de claves foráneas y transacciones.                         |
| <b>Honeypot</b>            | Campo oculto trampa para detectar y bloquear bots.                                     |
| <b>Path traversal</b>      | Ataque que intenta acceder a rutas fuera del directorio permitido.                     |
| <b>Seed</b>                | Datos de ejemplo insertados automáticamente al inicializar la BBDD.                    |
| <b>ngrok</b>               | Servicio que crea un túnel público hacia un servidor local.                            |
| <b>Debounce</b>            | Técnica que retrasa una acción hasta que el usuario deja de teclear.                   |
| <b>XAMPP</b>               | Paquete que reúne Apache, MySQL y PHP en un entorno local.                             |

## A.5 Tecnologías utilizadas



### Fin de la documentación

NEXUS · Foro Gaming Competitivo — Projecte Intermodular 2025-2026 · Hugo Reche. Documentación técnica completa del proyecto, su arquitectura, funcionalidades, seguridad, despliegue y planificación.